

Veci

Day 2 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“Find the smallest number larger than X consisting of the same digits as X ”*
- ▶ *“If there is no such number, output 00 ”*

Programming interpretation:

- ▶ The number of digits is small enough to try all digit permutations.
- ▶ Only permutations numerically larger than X are candidates.
- ▶ The smallest larger candidate is the requested answer.

Textbook connection: CP4 3.2 e. Iterative Complete Search (Permutation)

Core technique: Generate permutations and choose the smallest larger number

- ▶ We are given the digits of a number X .
- ▶ We need the smallest number larger than X that uses exactly the same digits.
- ▶ If no such number exists, print 0.

- ▶ The number of digits is small enough for permutation generation.
- ▶ Every valid answer is one permutation of the original digits.
- ▶ A min-heap makes it easy to retrieve the smallest candidate greater than X.

- ▶ Split the input number into characters.
- ▶ Generate all full-length permutations of those digits.
- ▶ Convert each permutation back to an integer.
- ▶ Push only values greater than the original number into a min-heap.
- ▶ Print the heap minimum, or 0 if the heap is empty.

- ▶ Duplicate digits may generate duplicate permutations, but duplicates do not affect the minimum.
- ▶ The heap stores only candidates that satisfy the greater-than requirement.
- ▶ The first pop is therefore the smallest valid rearrangement.

- ▶ All digit rearrangements are generated, so every feasible answer is considered.
- ▶ Filtering removes exactly the rearrangements that are not larger than X .
- ▶ The heap minimum among the remaining values is the required smallest larger number.

Complexity: $O(D * D! + D! \log(D!))$ time and $O(D * D!)$ memory for D digits.

- ▶ For larger digit counts, next-permutation would be better, but complete search is enough here.
- ▶ Converting through strings keeps the implementation short.

Source lines: veci.py:36-46

```
import sys
import itertools
import heapq

lines = sys.stdin.read().strip().splitlines()
ogNumber = int(lines[0])
numbers = [x for x in lines[0]]
candidateNumbers = []
permutations = [
    int("".join(x)) for x in list(itertools.permutations(numbers, len(
        numbers)))
]
```

What this chunk does

- ▶ The input number is kept both as an integer and as a list of characters.
- ▶ `itertools.permutations` enumerates all reorderings of those digits.
- ▶ Each tuple is joined and converted back to an integer candidate.

Source lines: veci.py:48-55

```
for permu in permutations:
    if permu > ogNumber:
        heapq.heappush(candidateNumbers, permu)

if not candidateNumbers:
    print(0)
else:
    print(heapq.heappop(candidateNumbers))
```

What this chunk does

- ▶ Only candidates larger than the original number enter the heap.
- ▶ A min-heap makes the smallest larger permutation available at the top.
- ▶ An empty heap means no valid larger number exists.

Solution file: `practice_contests/day_2/veci.py`

Problem: <https://open.kattis.com/problems/veci>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.